DAT 471 – Computational Techniques for Large-Scale Data

Module 6

Group 6

Name	Programme	Email
Gabriela Castillo Avila	MPHPC	avilag@student.chalmers.se
Mahnoor Ali Cheema	MPDSC	mahnoora@student.chalmers.se

27 May 2026

Problem 1 - Preprocessing

Before applying Locality-Sensitive Hashing (LSH), all word embeddings are normalized to unit length. Given an embedding matrix $X \in \mathbb{R}^{n \times d}$, each row vector x_i is transformed using L2 normalization:

$$\hat{x}_i = \frac{x_i}{\|x_i\|_2}.$$

The normalization was implemented using fully vectorized NumPy operations, avoiding explicit Python loops to ensure efficient computation on large datasets.
The experiment is executed on the 840B Common Crawl GloVe dataset (`glove.840B.300d.txt`), which contains approximately 2.2 million 300-dimensional word vectors. The total time required to normalize the dataset is:

34.1450982093811 seconds

Problem 2 - Matrix Multiplication

Nearest Neighbors for Query Words

Using the normalized GloVe 840B (300d) embeddings, we computed cosine similarities between the 10 query words and all vocabulary entries. For each query, the three closest neighbors (excluding the word itself) are shown below.

Query	1st Neighbor	2nd Neighbor	3rd Neighbor
priest	priests	bishop	Priest
fork	forks	spoon	Fork
horse	horses	pony	Horse
beef	pork	meat	chicken
daoist	taoist	daoism	confucian
polish	nail	polishes	nails
vehicle	vehicles	car	automobile
crepe	Crepe	chiffon	crêpe
daytime	nighttime	day-time	night-time
scotland	wales	glasgow	scottish

Table 1: Cosine similarities for 10 queries

Matrix Multiplication Performance

In order to compute cosine similarities between the 10 query vectors and all word embeddings in the GloVe 840B (300d) dataset, we formed a $10 \times d$ query matrix Q and evaluated the matrix product $QX^\top$, where X is the normalized $n \times d$ embedding matrix. The measured runtimes on Minerva were:

- **Matrix multiplication:** 0.2081 seconds
- **Sorting (nearest neighbors):** 1.1396 seconds

- **Total time:** 1.3478 seconds

These results demonstrate that the matrix multiplication step is extremely efficient due to NumPy's optimized BLAS backend, while the sorting step dominates the runtime because it must process all n similarity scores for each query.

Problem 3 - Hyperplanes

To reduce the dimensionality of the normalized GloVe embeddings, we applied the random hyperplane method for locality-sensitive hashing. For a chosen parameter D, we sampled D random Gaussian vectors $r_1, \ldots, r_D \in \mathbb{R}^d$ with entries drawn independently from $\mathcal{N}(0, 1)$, and normalized each vector to unit length:

$$\hat{r}_i = \frac{r_i}{\|r_i\|_2}.$$

Stacking these vectors produced a matrix $R \in \mathbb{R}^{D \times d}$. Each embedding vector $x \in \mathbb{R}^d$ was then projected via

$$x' = Rx,$$

and converted into a binary signature by thresholding:

$$x''_i = \begin{cases} 1 & \text{if } x'_i > 0, \\ 0 & \text{otherwise.} \end{cases}$$

Using the full Common Crawl GloVe dataset (840B, 300d) and $D = 50$ hyperplanes, the total time required to perform the transformation was:

2.31 seconds

This includes drawing the hyperplanes, projecting all vectors, and computing the final binary signatures.

Problem 4 - LSH

For the implementation of the *class LocalitySensitiveHashing* was implemented using the *class RandomHyperplanes* and function *normalize(X)* from previous problems.
running the program with the line:

```
python3 assignment6_problem4.py -D 50 -k 20 -L 10
/data/.../glove/glove.840B.300d.txt queries.txt
```

Gave the following results

Query	1st Neighbor	2nd Neighbor	3rd Neighbor
priest	deacon	shaman	congregation
fork	forks	spoons	wishbone
horse	sled	breeder	llama
beef	meats	roast	burgers
daoist	hellenistic	invlolved	two-handers
polish	nail	stain	pedicure
vehicle	off-road	roadway	roads
crepe	brunch	flour	cooked
daytime	hour-long	drowsy	dizziness
scotland	scottish	britain	manchester

The measured runtimes on Minerva were:

- **time to perform 10 queries:** 0.0434 seconds
- **Time to fit data:** 58.7238 seconds

Hyperparameter Search

Experimenting by variating the parameters D, k and L as following gave the results:
Variation of the first parameter

```
-D 50 -k 10 -L 10
```

Query	1st Neighbor	2nd Neighbor	3rd Neighbor
priest	Priest	deacon	vicar
fork	forks	skewer	tongs
horse	horses	equine	donkey
beef	meat	steak	meats
daoist	confucianism	atropa	buddist
polish	polishes	manicure	Polish
vehicle	vehicles	automobile	cars
crepe	chiffon	blouse	skirt
daytime	Daytime	primetime	naps
scotland	wales	scottish	edinburgh

Table 2: Parameters -D 50 -k 10 -L 10

The measured runtimes on Minerva were:

- **time to perform 10 queries:** 1.9577 seconds
- **Time to fit data:** 35.7909 seconds

Variation of the second parameter

```
-D 50 -k 20 -L 20
```

Query	1st Neighbor	2nd Neighbor	3rd Neighbor
priest	deacon	archbishop	church
fork	forks	spoons	wishbone
horse	gelding	foal	colt
beef	meats	roast	burgers
daoist	hellenistic	invlolved	two-handers
polish	nail	gloss	lacquer
vehicle	vehicles	off-road	roadway
crepe	omelette	gingham	corduroy
daytime	hour-long	drowsy	bedtimes
scotland	scottish	ireland	aberdeen

Table 3: Parameters -D 50 -k 20 -L 20

The measured runtimes on Minerva were:

- **time to perform 10 queries:** 0.0605 seconds
- **Time to fit data:** 115.7017 seconds

Variation of the third parameter

Query	1st Neighbor	2nd Neighbor	3rd Neighbor
priest	celebrant	confession	heretic
fork	20cm	crankarm	25cm
horse	equitation	horse-drawn	llama
beef	mignon	pureed	make
daoist	sin.I	invlolved	LaVeyan
polish	lacquer	glaze	primer
vehicle	tow	motorized	motorcar
crepe	tunic	muslin	casserole
daytime	drowsy	fleeting	Saturday-night
scotland	aberdeen	baltimore	augusta

Table 4: Parameters -D 60 -k 20 -L 10

The measured runtimes on Minerva were:

- **time to perform 10 queries:** 0.0255 seconds
- **Time to fit data:** 60.6931 seconds

Combination of all parameter changes

Query	1st Neighbor	2nd Neighbor	3rd Neighbor
priest	priests	Priest	ordained
fork	forks	Fork	bend
horse	horses	pony	Horse
beef	pork	chicken	steak
daoist	taoist	wudang	atropa
polish	nail	polishes	nails
vehicle	vehicles	car	automobile
crepe	crepes	silk	tulle
daytime	night-time	evenings	mornings
scotland	wales	glasgow	scottish

Table 5: Parameters -D 60 -k 10 -L 20

The measured runtimes on Minerva were:

- **time to perform 10 queries:** 6.0339 seconds
- **Time to fit data:** 69.0575 seconds

As we can see in table 2 lowering k gives fairly good semantics as can be see in the following examples:
vehicle → vehicles → automobile → car
beef → meat → steak → meats

Although the query time increases due to more vectors colliding while increasing the accuracy.

In table 3 we can identify semantically related neighbors, although farther away than the ground truths seen in table 1 for example:
vehicle → vehicle →s off-road → roadway

compared to:
vehicle → vehicles → car → automobile
Query time becomes much faster in this case as fewer collisions occur deecreasing the accuracy.

in table 4 neighbors become strange as seen in the example:
fork → 20cm → crankarm → 25cm

The increase in the parameter D in combination with a high k resulted in a fast query at the cost of accuracy, due to a fragmented bucket, resulting in fewer neighbors in the same bucket colliding.

A combination of the 3 parameter changes, results in closer neighbors as can be seen in table 5, the tradeoff lying in the high increase in both query and fitting data times.
These experiments clearly demonstrate the tradeoffs between accuracy and computational cost. Small k increases the bucket resulting in larger collidation while increasing the query

time, increase L improves the findings, due to more hash tables being searched resulting in increases the query time. And increased D improves the binary embeddings at the cost of the computational time.